

Resumen Ejecutivo de Calidad de Código

Sistema "Espaguetti de Encuestas"

Fase 3.2 --- Principios Clean Code | Arquitectura de Software

Encabezado

 Escenarios evaluados: 5

 Principios Clean Code aplicados:\

- Nombres significativos\
- Funciones pequeñas\
- Responsabilidad única\
- DRY\
- Manejo correcto de errores\
- Código autoexplicativo

 Principios SOLID relacionados:

SRP · OCP · DIP · ISP

 Stack: Spring Boot + Angular

ESCENARIO 1 --- Nombres Significativos

Anti-patrón: Uso de nombres ambiguos (r, resp, e, tmp)

ANTES

```
Map r = new HashMap();
r.put("id", id);
r.put("pregunta", pregunta);
return r;
```

```
this.http.get(this.url + '/encuestas')
  .subscribe(r => {
    console.log(r);
  });
```

DESPUÉS

```
Map<String, Object> response = new HashMap<>();
response.put("id", surveyId);
response.put("pregunta", surveyQuestion);
return response;
```

```
this.encuestaService.obtenerTodas()
    .subscribe((response: SurveyDTO[]) => {
        console.log(response);
    });
```

Tabla Comparativa

Criterio Antes Después

Legibilidad Baja Alta Intención Implícita Explícita Autodocumentación No Sí Tipado Débil Fuerte

✗ Violados: SRP, ISP

✓ Aplicados: SRP, ISP

Madurez:  Bajo →  Alto

Calificación: 3/10 → 9/10

ESCENARIO 2 --- Funciones Pequeñas y Responsabilidad Única

Anti-patrón: Método con múltiples responsabilidades

ANTES

```
@PostMapping("/crear")
public Map crear(@RequestBody Map body) {
    try {
        if (body == null || body.get("pregunta") == null
            || body.get("pregunta").toString().trim().length() < 3) {
            return null;
        }

        String sql = "INSERT INTO encuestas(...) VALUES ('"
            + body.get("pregunta") + "', 0, 0)";
        jdbc().update(sql);

        return body;
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```

    }
    return null;
}

```

■ DESPUÉS

```

@PostMapping("/crear")
public ResponseEntity<EncuestaDTO> crear(
    @Valid @RequestBody CrearEncuestaRequest request) {

    EncuestaDTO response = encuestaService.crear(request);
    return ResponseEntity.ok(response);
}

```

```

@Service
public class EncuestaService {

    public EncuestaDTO crear(CrearEncuestaRequest request) {
        Encuesta encuesta = new Encuesta();
        encuesta.setPregunta(request.pregunta());
        encuesta.setSiCount(0);
        encuesta.setNoCount(0);

        Encuesta saved = encuestaRepository.save(encuesta);
        return mapToDTO(saved);
    }
}

```

📊 Tabla Comparativa

Criterio Antes Después

Tamaño del método Largo Pequeño Responsabilidades Múltiples Única Testabilidad Baja Alta Cohesión Baja Alta

✗ **Violados:** SRP, DIP

☑ **Aplicados:** SRP, DIP

Madurez: ● Bajo → ● Alto

Calificación: 2/10 → 9/10

🔍 ESCENARIO 3 --- DRY (Don't Repeat Yourself)

Anti-patrón: Validaciones repetidas

■ ANTES

```
if (pregunta == null || pregunta.trim().length() < 3) {  
    throw new RuntimeException("Pregunta inválida");  
}
```

■ DESPUÉS

Backend --- Validación Declarativa

```
public record CrearEncuestaRequest(  
    @NotBlank(message = "La pregunta no puede estar vacía")  
    @Size(min = 3, message = "Mínimo 3 caracteres")  
    String pregunta  
) {}
```

Frontend --- Centralización

```
private validarPregunta(pregunta: string): boolean {  
    return pregunta != null && pregunta.trim().length >= 3;  
}
```

■ Tabla Comparativa

Criterio Antes Después

Repetición Alta Eliminada Consistencia Baja Alta Mantenibilidad Baja Alta

✗ **Violados:** DRY, SRP

☑ **Aplicados:** DRY, SRP

Madurez: ● Bajo → ● Alto

Calificación: 3/10 → 9/10

🔍 ESCENARIO 4 --- Manejo Correcto de Errores

■ ANTES

```
try {  
    return jdbc().queryForMap(sql);  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

```
}
return null;
```

■ DESPUÉS

```
public Encuesta buscarPorId(Long id) {
    return encuestaRepository.findById(id)
        .orElseThrow(() -> new EncuestaNotFoundException(id));
}
```

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(EncuestaNotFoundException.class)
    public ResponseEntity<String> handleNotFound(EncuestaNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

■ Tabla Comparativa

Criterio Antes Después

Retorno null Sí No Código HTTP Incorrecto Correcto Estandarización Nula Centralizada

✗ **Violados:** SRP, OCP

✓ **Aplicados:** SRP, OCP

Madurez: ● Bajo → ● Alto

Calificación: 2/10 → 9/10

🔍 ESCENARIO 5 --- Comentarios y Código Autoexplicativo

■ ANTES

```
// Crear encuesta nueva
Encuesta e = new Encuesta(); // instancia
e.setSiCount(0); // inicializa contador
e.setNoCount(0); // inicializa contador
```

 DESPUÉS

```
Encuesta encuesta = new Encuesta();
encuesta.setPregunta(request.pregunta());
encuesta.setSiCount(0);
encuesta.setNoCount(0);
```

 Tabla Comparativa

Criterio Antes Después

Comentarios redundantes Sí Eliminados Claridad semántica Baja Alta Expresividad Baja Alta

 **Violados:** Clean Code (claridad semántica)

 **Aplicados:** Código autoexplicativo

Madurez:  Bajo →  Alto

Calificación: 4/10 → 9/10

 TABLA RESUMEN GLOBAL

Escenario Anti-patrón Principio Antes Después Mejora

ESC-01 Nombres Nombres 3/10 9/10 +200% ambiguos significativos

ESC-02 Métodos largos SRP 2/10 9/10 +350%

ESC-03 Validación DRY 3/10 9/10 +200% repetida

ESC-04 return null Manejo correcto 2/10 9/10 +350% errores

ESC-05 Comentarios Código 4/10 9/10 +125% innecesarios autoexplicativo

 MÉTRICAS FINALES

- **Promedio ANTES:** 2.8 / 10\
- **Promedio DESPUÉS:** 9.0 / 10\
- **Mejora global:** +221%\
- **Nivel final de calidad:**  ALTO



Conclusión Técnica

La aplicación sistemática de principios Clean Code en la Actividad 3.2 transforma el sistema:

- De código funcional pero desorganizado\
- A una implementación clara, mantenible y profesional

Se fortalece:

- Legibilidad\
- Cohesión\
- Separación de responsabilidades\
- Robustez ante errores\
- Preparación para microservicios

El sistema evoluciona de un estado de madurez baja a un nivel alto de calidad interna, alineado con las prácticas promovidas por Robert C. Martin y los principios SOLID.